CS4371

Foundations of Big Data Management and Analytics

Fall 2026

Group Project 1

Spark Streaming with Kafka and Text Classification with Spark MLlib

Submission Deadline: November 15, 2026 (11:59 PM)

Instructions

- This assignment requires you to write code using Structured Streaming and MLlib libraries. To be able to run your code, you would need Apache Spark and other libraries mentioned installed in your environment. If you choose to run it on your local machine, all paths should be inputted as parameters. You cannot hardcode paths to your local computer.
- Teams have 3 or 4 students. Write the team number and the names and NetIDs of all members on the first page of the report and include a contribution table saying who did what.
- API keys (NewsAPI, Finnhub or any other source) must be read from environment variables. Never put a real key in your code or your submission.
- Record a narrated screen video of at most 8 minutes showing both parts of your project running, with every member explaining the part they built. Do not upload the video file. Put its link on the first page of the report and make sure the TAs can open it without requesting access.
- If you use AI assistants such as ChatGPT, Claude, Gemini or Copilot, add an appendix to the report listing each tool, who used it and for what, and submit the full chat transcripts as PDFs in a folder named ai_transcripts. Undisclosed AI assistance may be handled under the course's academic integrity policy.
- One member submits a single ZIP archive named <TeamNumber>-GP1.zip containing everything listed under What to Submit in Parts 1 and 2, the report as a PDF and the ai_transcripts folder.

1 Spark Streaming with Real Time Data and Kafka

In this part, you will create a Spark Streaming application that will continuously read text data from a real time source, analyze the text for named entities, and send their counts to Apache Kafka. A pipeline using Elasticsearch and Kibana will read the data from Kafka and analyze it visually.

Setting up your development environment

You can set up everything on your local machine or host them on the cloud. Following are the resources that you will need:

- You will need to get access to a real-time text data source. Some examples are:
  - NewsAPI - You can download real time news using the News API available at: https://newsapi.org/ . There is also a Python wrapper for this available at: https://newsapi.org/docs/client-libraries/python
  - Finnhub For financial news and analysis, the Finnhub library https://finnhub.io/ can be used. There is a Python wrapper for this available at https://pypi.org/project/finnhub-python/ .
  - Bluesky Jetstream - You can receive public Bluesky posts in real time over a WebSocket connection at: https://bsky.network/docs/jetstream/ . It is free and needs no account or API key. The post text is in the commit.record.text field of each JSON message. The volume is high, so you may want to keep only English posts (the langs field) or a sample.

  - Hacker News API - You can get new stories and comments from Hacker News using its free API at: https://github.com/HackerNews/API . No key is needed. Poll https://hacker-news.firebaseio.com/v0/newstories.json (or maxitem.json) at periodic intervals and fetch each new item. Comment text is in HTML, so remove the tags before extracting named entities.
- Download Apache Kafka and go through the quickstart steps: https://kafka.apache.org/quickstart
- Windows users might want to consider WSL, which gives a Unix-like environment on Windows: https://docs.microsoft.com/en-us/windows/wsl/install-win10
- You will need a working Spark cluster. It can be setup locally or you can use a cloud based server.
- I have attached a simple project showing how to pass data between Kafka and a Spark Structured Streaming application that performs word count. Instructions for running are also attached.
- Later, you will also need to set up Elasticsearch and Kibana environment to visualize data. You will need to download Elasticsearch, Kibana, and Logstash from https://www.elastic.co/downloads/

Project Steps

For this project, you will need to perform the following steps:

- Create a Python application that reads from a real-time data source, such as the ones mentioned in the previous part. Some libraries have a streaming version that continuously fetch real-time data. For others, you might have to write a loop that gets real time data at periodic intervals.
- This incoming data should continuously be written to a Kafka topic (let's call it topic1 for illustration).
- Create a PySpark structured streaming application that continuously reads data from the Kafka topic (topic1) and keeps a running count of the named entities being mentioned. You should already know how to extract named entities from text. In this part, you will keep a running count of the named entities being mentioned.
- At the trigger time, a message containing the named entities and their counts should be sent to another Kafka topic (let's call it topic2 for illustration).
- Configure Logstash, Elasticsearch, and Kibana to read from the Kafka topic (topic2) and create a bar plot of the top 10 most frequent named entities and their counts.

Visualizing the data using Elasticsearch and Kibana

You will need Elasticsearch and Kibana installed to get the data from Kafka and visualize it. Details are available at:

https://www.elastic.co/downloads

https://www.elastic.co/docs/get-started

You will need to visualize the count of top 10 named entities being mentioned on your chosen data source. You should be able to view this using a barplot in Kibana.

What to Submit

You are required to submit the following:

- Your project code file.
- Named entity frequency bar plots taken at several intervals. For example, you can show the top 10 named entities being mentioned after 15, 30, 45, and 60 minutes.
- A report explaining your data source and also a small paragraph explaining what your results indicate.
- A README file indicating how to run your code. Please do not use any hard coded paths.

2 Text Classification of Streamed Text with PySpark

In this part, you will use a text classifier built with Spark MLlib to label the live text you collected in Part 1 by topic. Your streaming data has no labels, so you will fit the classifier on a labeled news dataset, AG News, and then apply it to your own messages, first as a batch and then in real time inside your streaming application. AG News has four classes: World, Sports, Business and Sci/Tech. It is available for free at https://huggingface.co/datasets/fancyzhx/ag_news (Parquet files; use the training file, which has 120,000 articles with the columns text and label).

2.1 Loading Data

Load the AG News training file into a Spark DataFrame and report how many articles each class has. Then load the text you streamed in Part 1: read every message from topic1 with a batch Kafka read (spark.read.format("kafka") with startingOffsets set to earliest) or from the file your producer saved, and keep only the message text. You should have at least 1,000 streamed messages. Remove empty and duplicate messages and report how many remain.

2.2 Building the Pipeline

Build an MLlib Pipeline that turns raw text into a predicted class. It should at least contain the following stages:

a. RegexTokenizer to split the text into lowercase words and drop punctuation.
b. StopWordsRemover to remove common words such as “the” and “and”.
c. HashingTF or CountVectorizer followed by IDF to produce TF-IDF features. You are free to define any suitable parameters, such as the number of features or the minimum document frequency.
d. LogisticRegression as the last stage.

Fit the Pipeline on the AG News training data and save the fitted PipelineModel to a path given as a parameter. Use this same model for everything that follows, so your streamed text goes through exactly the same steps.

2.3 Classifying Your Streamed Data

Load the saved PipelineModel and use it on the text from Part 1:

a. Batch: predict a class for every streamed message and report how many messages fall in each class. Show five example messages for each class with their predicted probability.
b. Check by hand: pick 50 streamed messages at random, label each one yourself as World, Sports, Business, Sci/Tech or None of these, and compare your labels with the model’s predictions. Report the accuracy on the messages that fit one of the four classes, and how many fit none.

c. Real time: add a second query to your Part 1 Structured Streaming application that reads topic1, applies the saved model to each incoming message, and keeps a running count of messages per predicted class. Show these counts in the console, or send them to another Kafka topic, at each trigger.

2.4 What to submit

You are required to submit the following:

- Your project code file.
- Output file with the predicted class and probability for every streamed message, and a file with your 50 hand labels next to the model's predictions.
- A brief summary indicating whether the predicted classes make sense for your data source, which kinds of messages the model handles well or badly and why, and a few examples of messages it got wrong.
- A README file indicating how to run your code. Please do not use any hard coded local paths.

Good luck!